

Grafični vmesniki v programskem jeziku Java

Splošnonamenski programski jezik **Java** je bil razvit kot **varen jezik za poljubno napravo** (preverjanje tipov, vidljivost konstruktov, virtualni stroj itd.). Sintaksa zahteva daljše programe kot v programskem jeziku Python, ki pa so veliko hitrejši. Jezik se prevaja, kar pomeni, da program `GUI2.java` v ukazni vrstici najprej prevedemo z `javac GUI2.java`, kar ustvari vmesno datoteko `GUI2.class`, katero nato izvedemo z `java GUI2`.

...

Animacija likov in grafičnih oblik

Geometrijske **like** in grafične **oblike lahko animiramo** tako, da pri izrisu upoštevamo stanje izbranih (zunanjih) spremenljivk, ki jih na primer posodablamo v določenih časovnih intervalih.

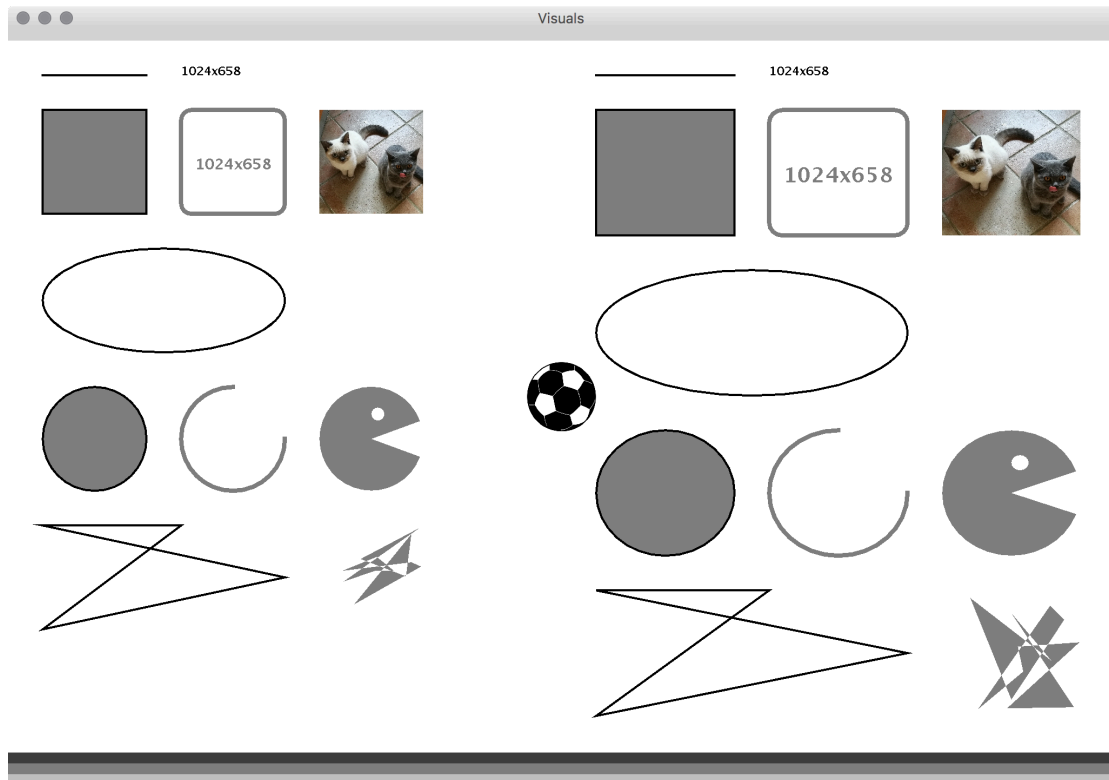
Spodnji program doda izris slike nogometne žoge, katere središče je predstavljeno z javno statično spremenljivko `ball` tipa `java.awt.Point` definirano v razredu `GUI2`.

```
public class GUI2 {  
  
    public static Point ball = new Point();  
  
    public static void main(String[] args) {  
        // ...  
        ball = new Point(panel.getWidth() / 2, panel.getHeight() / 2);  
        // ...  
    }  
  
}  
  
class Panel2 extends JPanel {  
  
    @Override  
    public void paint(Graphics g) {  
        // ...  
        try { // shrani sliko v spremenljivko ali konstanto!  
            graphics.drawImage(ImageIO.read(new File("images", "football  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
  
}
```

Spodnji program doda še animacijo nogometne žoge, ki se odbija od stranic panela po katerem rišemo.

```
public class GUI2 {  
  
    public static void main(String[] args) {  
        // ...  
        Point direction = new Point(8, -8);  
        while (true) {  
            ball.setLocation(ball.getX() + direction.getX(), ball.getY() + direction.getY());  
            if (ball.getX() <= 32) {  
                ball.setLocation(32, ball.getY());  
                direction.setLocation(-direction.getX(), direction.getY());  
            }  
            else if (ball.getX() >= panel.getWidth() - 32) {  
                ball.setLocation(panel.getWidth() - 32, ball.getY());  
                direction.setLocation(-direction.getX(), direction.getY());  
            }  
            if (ball.getY() <= 32) {  
                ball.setLocation(ball.getX(), 32);  
                direction.setLocation(direction.getX(), -direction.getY());  
            }  
            else if (ball.getY() >= panel.getHeight() - 32) {  
                ball.setLocation(ball.getX(), panel.getHeight() - 32);  
                direction.setLocation(direction.getX(), -direction.getY());  
            }  
            frame.repaint(); // ponoven izris okna  
            try {  
                Thread.sleep(50); // počakaj 50 ms  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
    }  
}
```

Primer izgleda grafičnega vmesnika je prikazan spodaj.



Grafični elementi in nadzor programa

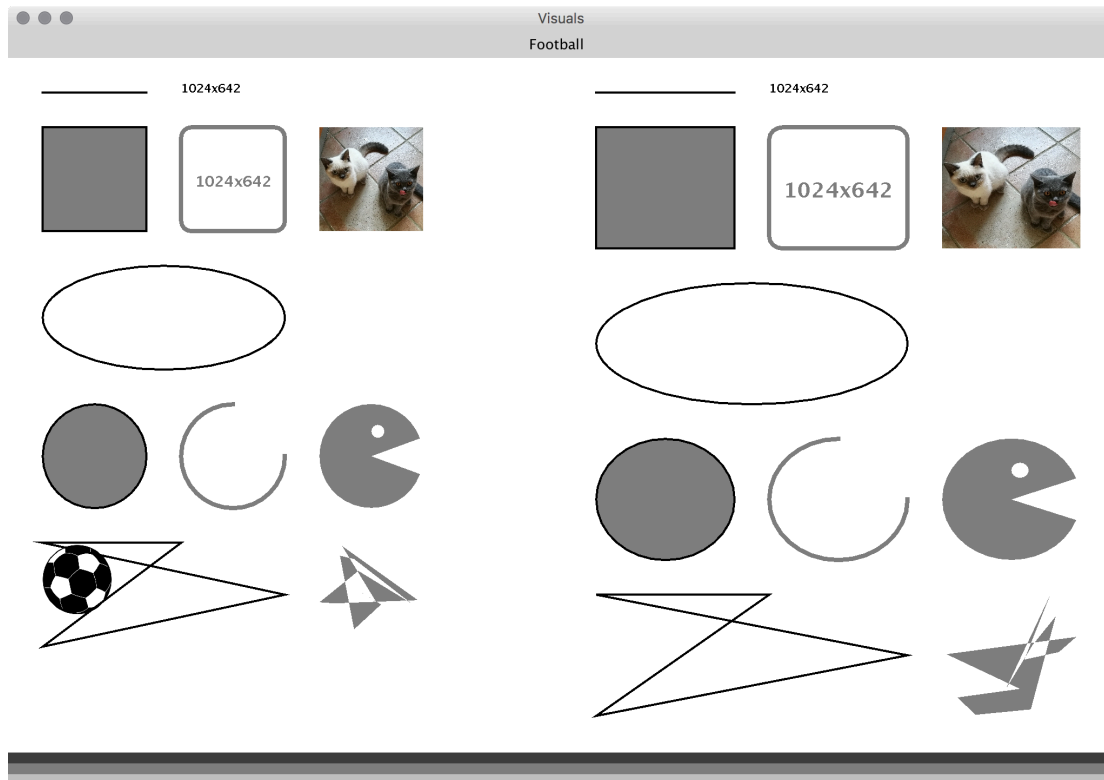
Panelom razreda `JPanel` lahko **dodamo različne grafične elemente** z uporabo metode `add(Component component)`. Med **osnovne grafične elemente** štejemo labelo razreda `JLabel`, gumbe razredov `JButton` in `JToggleButton`, drsnike razreda `JSlider`, vnosna polja razreda `JTextField`, izbirnike razredov `JCheckBox` in `JComboBox` itd. v paketu `javax.swing`.

Spodnji program **doda labelo** razreda `JLabel` severnemu panelu programskega okna.

```
north.add(new JLabel("Football  "));
```

Java

Primer izgleda grafičnega vmesnika je prikazan spodaj.



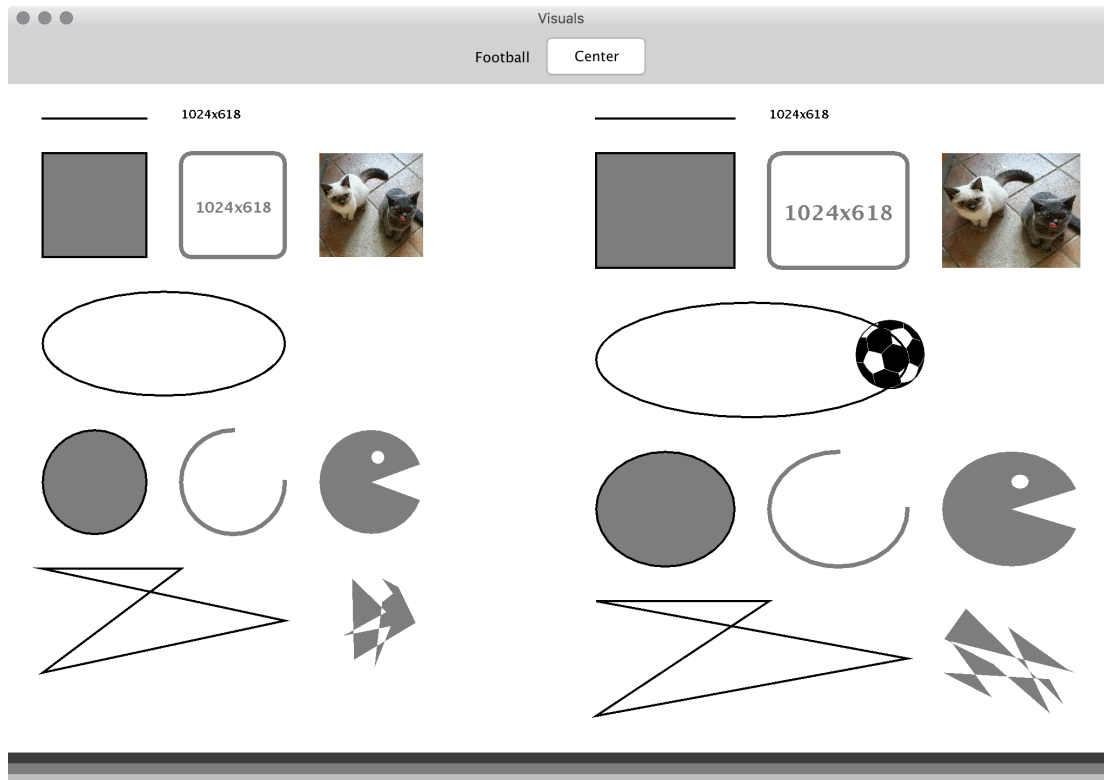
Grafični elementi **preko dogodkov** (npr. pritisk gumba, sprememba drsnika) **omogočajo nadzor** nad delovanjem programa oziroma grafičnega vmesnika. Na same dogodke se odzovemo tako, da grafičnemu elementu **dodamo poslušalca**, ki implementira vmesnik `java.util.EventListener` in s tem vse potrebne metode za odziv na dogodke grafičnega elementa.

Spodnji program **doda gumb** razreda `JButton` severnemu panelu programskega okna. Gumbu **dodamo poslušalca**, ki implementira vmesnik `java.awt.event.ActionListener` kot anonimni razred in ob kliku na gumb nogometno žogo prestavi nazaj v središče osrednjega panela.

```

JButton button = new JButton("Center");
button.setPreferredSize(new Dimension(96, 40));
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        ball = new Point(panel.getWidth() / 2, panel.getHeight() / 2);
    }
});
north.add(button);
  
```

Primer izgleda grafičnega vmesnika je prikazan spodaj.



Spodnji program **doda drsnik** razreda `JSlider` severnemu panelu programskega okna, ki določa hitrost posodabljanja središča nogometne žoge. Drsniku **dodamo poslušalca**, ki implementira vmesnik `java.awt.event.ChangeListener` in ob spremembi zahteva ponoven izris osrednjega panela.

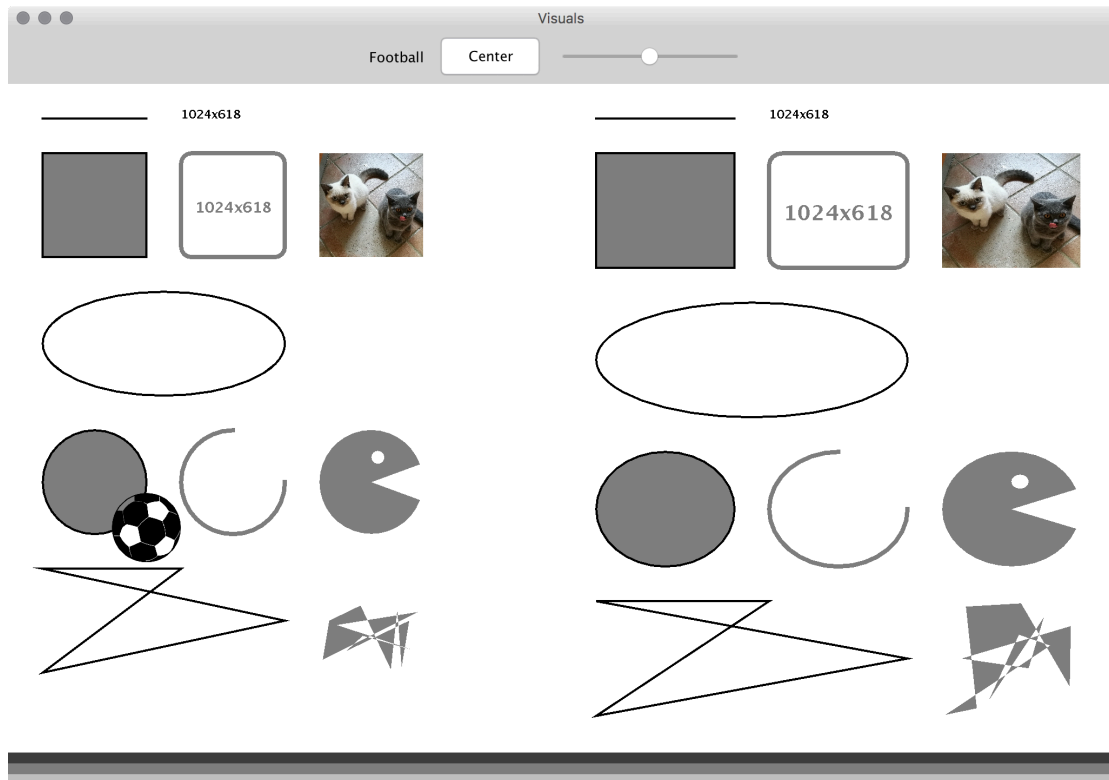
```

JSlider slider = new JSlider(1, 100, 50);
slider.addChangeListener(new ChangeListener() {
    @Override
    public void stateChanged(ChangeEvent e) {
        panel.repaint();
    }
});
north.add(slider);
// ...
while (true) {
    // ...
    try {
        Thread.sleep(slider.getValue());
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

```

Java

Primer izgleda grafičnega vmesnika je prikazan spodaj.



Spodnji program **doda izbirnik** razreda `JCheckBox` severnemu panelu programskega okna, ki nadzira posodabljanje središča nogometne žoge. Izbirniku **dodamo poslušalca**, ki implementira vmesnik `java.awt.event.ActionListener` in ob spremembi zahteva ponoven izris osrednjega panela.

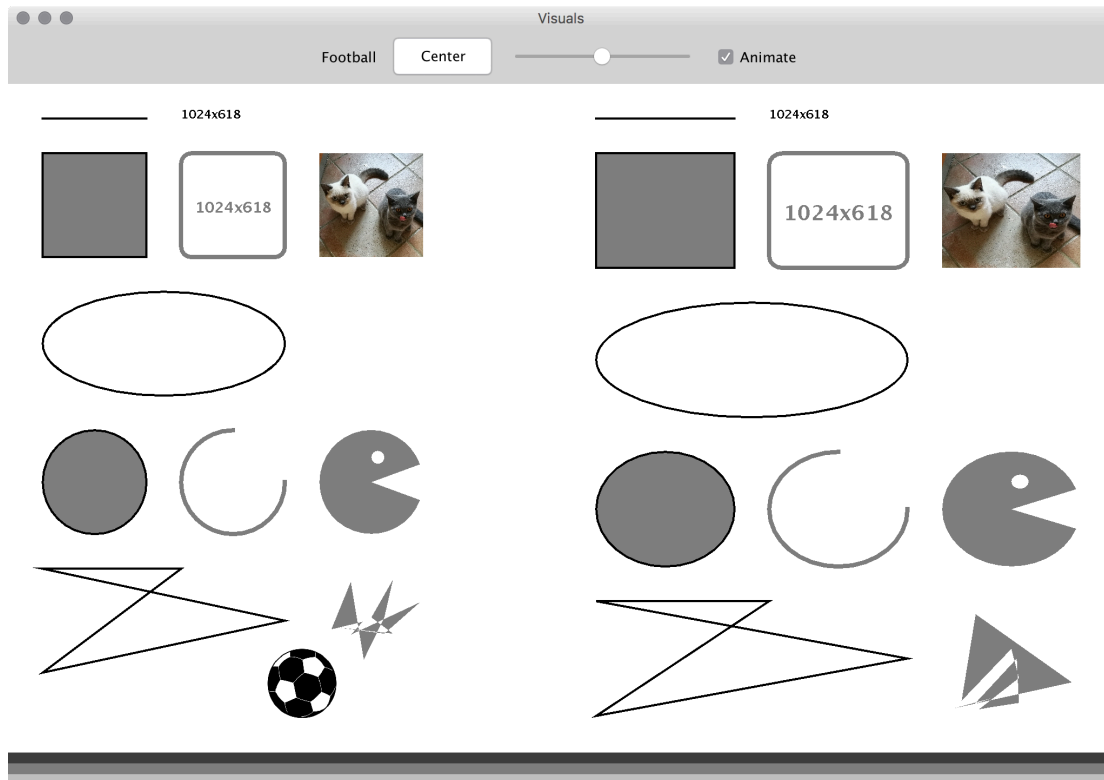
```

JCheckBox checkbox = new JCheckBox("Animate", true);
checkbox.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        panel.repaint();
    }
});
north.add(checkbox);
// ...
while (true) {
    if (checkbox.isSelected()) {
        // ...
    }
    // ...
}

```

Java

Primer izgleda grafičnega vmesnika je prikazan spodaj.



Spodnji program **doda vnosno polje** razreda `JTextField` severnemu panelu programskega okna, ki ob pritisku na vnašalko izbriše vsebino vnosnega polja. Vnosnemu polju **dodamo poslušalca**, ki implementira vmesnik `java.awt.event.KeyListener` in se izvede ob pritisku na poljubno tipko.

```

JTextField field = new JTextField();
field.setPreferredSize(new Dimension(128, 24));
field.addKeyListener(new KeyListener() {
    @Override
    public void keyPressed(KeyEvent e) {
        if (e.getKeyCode() == KeyEvent.VK_ENTER)
            field.setText("");
    }
    @Override
    public void keyReleased(KeyEvent e) { }
    @Override
    public void keyTyped(KeyEvent e) { }
});
north.add(field);

```

Java

Primer izgleda grafičnega vmesnika je prikazan spodaj.

